

Crisis Monitor v1.1

Documentación del sistema · Sistema de detección de crisis en redes sociales

Sistema de detección de crisis en redes sociales mediante análisis de sentimiento en español. Clasifica comentarios en cinco categorías, genera alertas configurables y persiste los resultados en cuatro formatos.

Plataforma: Python 3.8+ — solo biblioteca estándar (sin dependencias externas) · **Interfaz:** menú interactivo de consola · **Clasificación:** reglas léxicas con diccionario ponderado en español (256 entradas, incluye jerga peruana e intensificadores) — sin modelos preentrenados de IA.

Inicio rápido

```
python main.py
```

Dos caminos:

- **Dataset del proyecto:** elige la opción `1` y escribe `comentarios_x.csv` (69 comentarios en español que simulan publicaciones de la plataforma X). Tienes tu primer reporte de crisis en menos de 2 minutos.
- **Demo express:** elige la opción `9` para generar un CSV de ejemplo de 16 filas (`comentarios_ejemplo.csv`) y luego `1` + Enter para analizarlo.

Importante: la opción 9 sobrescribe cualquier archivo llamado `comentarios_ejemplo.csv` . Por eso el dataset oficial del proyecto se llama `comentarios_x.csv` — no lo renombres a `comentarios_ejemplo.csv` .

Dataset del proyecto

El sistema acepta cualquier CSV con las columnas `id` , `texto` , `fecha` . La columna `usuario` es **opcional**: si está presente, se usa en la persistencia, en las alertas y en la sección "usuarios con más comentarios negativos" del reporte; si no existe, queda vacía sin generar error.

El dataset oficial es `comentarios_x.csv` : 69 comentarios en español que simulan publicaciones de X, contruidos a partir de casos públicos documentados en redes sociales peruanas. Incluye deliberadamente duplicados y textos con ruido (símbolos, enlaces, jerga) para demostrar el proceso de limpieza, deduplicación y filtrado.

Para volúmenes mayores, la utilidad `convertir_kaggle.py` adapta el dataset público 'NLP with Disaster Tweets' de Kaggle (`train.csv` , 7 613 filas) al formato del sistema.

La recolección automática desde X queda fuera del alcance (la API oficial eliminó su tier gratuito en 2023 y el scraping no autorizado infringe los términos de servicio de la plataforma). Ver *Arquitectura del sistema* para la decisión completa.

Módulos

Módulo	Responsabilidad
<code>main.py</code>	Punto de entrada; menú interactivo de 9 opciones
<code>ingesta.py</code>	Leer CSV, limpiar texto, detectar duplicados, filtrar ruido
<code>clasificador.py</code>	Análisis de sentimiento con léxico ponderado en español (POO con herencia: <code>AnalizadorBase</code> → <code>AnalizadorSentimiento</code>)
<code>alertas.py</code>	Alertas individuales y alerta de crisis global
<code>persistencia.py</code>	Guardar y leer en TXT, CSV, Pickle y SQLite (DML completo)
<code>reportes.py</code>	Reporte de crisis en texto y resumen en consola
<code>convertir_kaggle.py</code>	Utilidad: convierte el <code>train.csv</code> de Kaggle al formato del sistema

Documentación

Documento	Tipo	Descripción
Tutorial: tu primer análisis	Tutorial	Desde cero hasta tu primer reporte de crisis
Cómo hacer tareas comunes	How-to	Cargar CSV, configurar umbrales, exportar datos, usar Kaggle
Referencia de módulos	Referencia	API completa: parámetros, tipos, retornos, ejemplos
Arquitectura del sistema	Explicación	Por qué el sistema funciona como funciona

Flujo de datos

```
CSV de entrada → ingesta → clasificador → alertas → persistencia → reportes
```

Cada etapa tiene responsabilidad única. Ver *Arquitectura* para el diagrama completo y las decisiones de diseño.

Salidas generadas

Después de cargar un CSV con la opción **1**:

```
datos_salida/
├── comentarios.txt      + todos los comentarios con categoría y puntuación
├── negativos.csv        + solo comentarios negativos/muy negativos (incluye usuario)
└── datos.pkl           + backup completo (datos + alertas)

crisis.db               + base de datos SQLite con historial completo
                        (creada con schema.sql: tablas comentarios y alertas)

datos_salida/reporte_crisis.txt + reporte de crisis (generado con opción 5)
```

La base de datos se gestiona con los cuatro verbos DML: INSERT, SELECT, UPDATE y DELETE (demo en vivo con la opción **7** del menú).

Resultados de referencia (dataset del proyecto)

Métrica	Valor
Filas leídas	69
Duplicados descartados	26
Ruido descartado (texto < 3 caracteres)	2
Comentarios únicos analizados	41
Índice de crisis	36.6% (15/41 negativos) → nivel ALTO
Alertas	10 individuales + 1 crisis global = 11

Umbrales por defecto

Parámetro	Valor	Configurable
Umbral de alerta individual	-2.0	Sí (opción 8)
% de negativos para crisis global	30%	No (en <code>main.py</code>)
Límite de crisis ALTO	30%	No (en <code>reportes.py</code>)
Límite de crisis CRITICO	60%	No (en <code>reportes.py</code>)